

Step 14: Building the Frontend and Flask Web Application

The deployment phase of the Credit Card Approval Prediction System involves developing a web-based application that allows users to interact with the trained machine learning model through a simple graphical interface. The application is developed using HTML, CSS, and the Flask web framework. Flask acts as the communication bridge between the frontend user interface and the backend machine learning model, enabling real-time credit card approval prediction.

Step 14.1: Creating the Project Directory

The project directory is organized to separate the frontend, backend, and machine learning model files. A **templates** folder is created to store all HTML pages, while the trained model and Flask application script are stored in the project root directory.

Example Project Structure:

```
Credit_Card_Approval_Prediction/  
|  
├─ app.py  
├─ model.pkl  
├─ templates/  
|   ├─ home.html  
|   ├─ index.html  
|   └─ result.html  
├─ static/  
|   ├─ style.css  
|   └─ images/  
└─ requirements.txt
```

Step 14.2: Creating the Templates Folder

A folder named **templates** is created because Flask automatically searches for HTML files inside this directory. All web pages required for the application are placed inside this folder.

The templates folder contains:

- **home.html**
- **index.html**

- **result.html**

Step 14.3: Developing the Home Page (home.html)

The **home.html** page is designed as the landing page of the application. It introduces the Credit Card Approval Prediction System, explains the objective of the project, and provides a navigation button that redirects users to the prediction form.

This page serves as the entry point of the web application and improves the overall user experience.

Step 14.4: Developing the Prediction Form (index.html)

The **index.html** page contains the applicant input form. Users enter all the financial, demographic, and credit-related information required by the trained machine learning model.

The form includes input fields for the selected features used during model training. After completing the form, users submit their information by clicking the **Predict** button.

Step 14.5: Developing the Result Page (result.html)

The **result.html** page displays the prediction generated by the machine learning model. After the applicant information is processed, the page clearly indicates whether the credit card application is **Approved** or **Rejected**.

The page may also provide an option to return to the prediction form for another evaluation.

Step 15: Building the Flask Backend Script

The backend of the application is developed using the Flask framework. The main Python script, **app.py**, controls the entire workflow, including routing, model loading, user input processing, prediction generation, and result display.

Step 15.1: Importing Required Libraries

The necessary Python libraries are imported to build the Flask application. Flask provides functions for creating web routes and rendering HTML pages, while Pickle or Joblib is used to load the trained machine learning model. Additional libraries such as NumPy and Pandas may also be used to process user input before prediction.

Step 15.2: Initializing the Flask Application

A Flask application object is created to manage all web requests. This object controls navigation between different pages, receives user input, and communicates with the machine learning model.

Step 15.3: Loading the Trained Machine Learning Model

The trained Credit Card Approval Prediction model is loaded from the saved **model.pkl** file using Pickle or Joblib. Loading the saved model eliminates the need to retrain the model each time the application is executed and enables faster prediction.

Step 15.4: Creating the Home Route

A route is created to display the **home.html** page when users open the application in their web browser. The page is rendered using Flask's `render_template()` function.

Step 15.5: Creating the Prediction Form Route

A separate route is created to display the **index.html** page. This page allows users to enter applicant details that will be used for prediction.

Step 15.6: Collecting User Input

When the user submits the prediction form, Flask receives the entered values through the request object. The input data is collected, validated, and converted into the numerical format expected by the machine learning model.

Proper input processing ensures compatibility between the user interface and the trained model.

Step 15.7: Generating the Prediction

The processed applicant data is passed to the trained machine learning model using the `predict()` method.

The model analyzes the applicant's financial and demographic information and predicts whether the credit card application should be approved or rejected.

Step 15.8: Displaying the Prediction Result

The prediction generated by the model is sent to the **result.html** page. Flask renders the page and displays the final prediction result in a clear and user-friendly format.

This allows users to instantly view the outcome of their credit card application.

Step 15.9: Running the Flask Application

Finally, the Flask development server is started. The application becomes accessible through a web browser, where users can navigate between the home page, prediction form, and result page.

The completed system provides a complete end-to-end workflow, allowing users to enter applicant details, process the information through the trained machine learning model, and receive real-time credit card approval predictions through an interactive web interface.